

Solidity Code Breakdown



```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract MyFavoriteNumber {
    uint public myNumber = 10;
    string public myStatus = "Learning Solidity";
    // This is our new function!
    function setMyNumber(uint _newNumber) public {
        myNumber = _newNumber;
    }

    function setMyStatus(string memory _myStatus) public {
        myStatus = _myStatus;
    }
}
```

Explain this code in details and answer these question in short 1. why memory keyword is used and relate with other like this ? 2. why string is dynamic and size is not fixed like uint is 256 bit ? 3. Why here used of _(underscore)?

Alright — let's break this down **step by step**.

I'll explain the **code** first, then answer your **3 questions in short** but clearly.

✓ Code Explanation

```
solidity
```

```
// SPDX-License-Identifier: MIT
```

- This line declares the **license type** (MIT).
It's good practice for open-source projects to clarify usage rights.

```
solidity
```

```
pragma solidity ^0.8.20;
```

- This tells the compiler which version of Solidity to use (**0.8.20 or higher** but less than 0.9.0).

```
solidity
```

```
contract MyFavoriteNumber {
```

- Defines a **contract** named **MyFavoriteNumber**.
A contract in Solidity is like a **class** in other languages.

```
solidity
```

```
uint public myNumber = 10;
```

- Declares a **public unsigned integer** named `myNumber` with an initial value `10`. **public** creates an automatic **getter function**, so anyone can read this value.

```
solidity
```

```
string public myStatus = "Learning Solidity";
```

- Declares a **public string** variable `myStatus` initialized with `"Learning Solidity"`.

```
solidity
```

```
function setMyNumber(uint _newNumber) public {
    myNumber = _newNumber;
}
```

- Defines a **public function** `setMyNumber` that lets anyone change `myNumber`.
- Takes an input `_newNumber` and assigns it to `myNumber`.

```
solidity
```

```
function setMyStatus(string memory _myStatus) public {
    myStatus = _myStatus;
}
```

- Defines a **public function** `setMyStatus` that lets anyone change `myStatus`.
- Takes a **string input** `_myStatus` which is stored in **memory** (temporary).
- Updates `myStatus` with this new string.

Why memory keyword is used?

- **memory** means the variable is stored **temporarily** while the function runs.
- Needed for **reference types** (like `string`, `array`, `struct`) to specify **where data lives**.
- Example:
 - `uint` or `bool` don't need **memory** because they are **value types** — copied directly.
 - `string` is a **reference type** — so Solidity needs to know: do you want it stored in **memory**, **storage**, or **calldata**?

Why string is dynamic and uint is fixed-size?

- `string` can have **any length** (1 char or millions) — it's a **dynamic array of bytes**.
- `uint` (or `uint256`) has a **fixed size** — always **256 bits** (32 bytes).

- So:
 - `uint` → size is known at compile time.
 - `string` → size is unknown until runtime.

Why use `_` (underscore) before parameter names?

- It's a **naming convention** to distinguish **function parameters** from **state variables**.
- Example:

```
solidity

function setMyNumber(uint _newNumber) public {
    myNumber = _newNumber;
}
```

Here:

- `myNumber` → state variable.
- `_newNumber` → function input.
- Helps avoid confusion if both have similar names.

Summary

memory	For temporary storage of reference types .
string	Dynamic size → not fixed like <code>uint</code> .
_ underscore	Good practice to clearly name function inputs.

If you'd like, I can also make **handy notes** or a **PDF** — just say "Yes!"



memory, storage, or calldata? give me little description about these

Great follow-up — let's break down `memory`, `storage`, and `calldata` in a **simple table + short notes**:

memory

- **Where?** → Temporary, lives only during function execution.
- **Who uses it?** → Reference types inside **functions** (e.g., `string` `memory`, `bytes` `memory`, `struct` `memory`).
- **Example:**

```
solidity
```

```
function setName(string memory _name) public { ... }
```

- **Key point:** Changes to `memory` don't affect blockchain state.

✓ 2 storage

- **Where?** → Permanent storage on the blockchain.
- **Who uses it?** → State variables by default live in **storage**.
- **Example:**

```
solidity
```

```
string public myName; // in storage
```

You can also create **references** to **storage** inside functions:

```
solidity
```

```
string storage myRef = myName;
```

- **Key point:** Changes **do** update the blockchain — costs **gas**.

✓ 3 calldata

- **Where?** → Temporary, **read-only**, for **function inputs** (external/public).
- **Who uses it?** → Input parameters for **external functions**, especially arrays & strings.
- **Example:**

```
solidity
```

```
function greet(string calldata _name) external { ... }
```

- **Key point:** More **gas efficient** for read-only input data — can't modify it.

✓ Summary Table

Keyword	Lifespan	Mutable?	Typical Use
memory	Temporary (in RAM)	✓ Yes	Inside functions for temp data
storage	Permanent (on chain)	✓ Yes	State variables, saved on blockchain
calldata	Temporary (read-only)	✗ No	Function inputs (external/public)

If you'd like, I can make a **cheat sheet** for this too — just say **"Make it!"** 📄 ✨